Lab 3 – Herencia: Análisis del sistema TecniHogar

CC2008 – Programación Orientada a Objetos · Ciclo 2, 2026

1. Requisitos funcionales

1.1 Información que el sistema debe conservar

Entidad	Información	Regla de validez
Orden de servicio (común)	Código, cliente, dirección, horas estimadas, zona, estado, visitas realizadas	Código, cliente y dirección no vacíos; código único en todo el sistema; horas estimadas > 0 (admite decimales); zona metropolitana o periférica
Diagnóstico	Solo los datos comunes	Horas estimadas ≤ 1.5
Instalación	Puntos a instalar	Entero positivo
Instalación eléctrica certificada	Los de una instalación + valor declarado de materiales	Valor de materiales > 0
Mantenimiento de aire acondicionado	Capacidad en BTU	BTU entre 9,000 y 60,000; solo zona metropolitana
Cuadrilla	Código, capacidad máxima de horas por jornada, certificación en refrigeración, si está libre	Código único y no vacío; capacidad > 0
Jornada	Código, cuadrilla, órdenes asignadas, resultado de cada orden, estado	Una cuadrilla no puede tener más de una jornada sin cerrar
Ingresos	Monto por registro y monto por visitas adicionales, ambos por categoría	Solo aumentan por registro de una orden o por inicio de una jornada

1.2 Operaciones y condiciones de validez

Operación	Qué hace	Condiciones de validez (si no se cumplen: se rechaza con mensaje y no cambia nada)
Cotizar	Muestra costo y tiempo estimado de una orden	Datos válidos según la categoría. No registra nada ni altera ingresos
Registrar orden	Guarda la orden como pendiente y	Datos válidos; el código no existe todavía; el cliente aceptó la

Operación	Qué hace	Condiciones de validez (si no se cumplen: se rechaza con mensaje y no cambia nada)
	cobra el monto completo	cotización
Registrar cuadrilla	Agrega una cuadrilla	Código no vacío y no repetido; capacidad > 0
Crear jornada	Crea una jornada en preparación para una cuadrilla	La cuadrilla existe y está libre
Asignar orden	Agrega una orden a una jornada	Jornada en preparación; orden pendiente (no asignada a otra jornada, ni completada ni cancelada); suma de horas estimadas ≤ capacidad de la cuadrilla; orden compatible con la cuadrilla
Retirar orden	Saca una orden de la jornada; vuelve a pendiente	Jornada en preparación; la orden pertenece a la jornada
Iniciar jornada	Pasa la jornada y sus órdenes a "en atención", cuenta una visita por orden y cobra Q40 por cada visita adicional a la primera	Jornada en preparación y con al menos una orden
Registrar resultado	Marca una orden como completada o con visita fallida; cierra la jornada cuando todas tienen resultado	Jornada en atención; la orden pertenece a la jornada y aún no tiene resultado en ella
Consultar	Lista órdenes, cuadrillas y jornadas	Ninguna
Reportes	Órdenes por categoría y estado; ingresos (por registro, por visitas adicionales y por categoría); jornadas no cerradas con sus horas asignadas y capacidad disponible	Ninguna. Son solo lectura

1.3 Reglas de cobro y tiempos

Categoría	Costo	Atención (metropolitana / periférica)	Visitas permitidas
Diagnóstico	Q150 metropolitana, Q225 periférica, sin importar las horas	1 / 2 días	3

Categoría	Costo	Atención (metropolitana / periférica)	Visitas permitidas
Instalación	Base (Q100 metropolitana, Q175 periférica) + Q90 × horas facturables. Horas facturables = techo de max(horas estimadas, puntos × 0.75)	3 / 5 días	3
Instalación eléctrica certificada	Costo de instalación + Q120 de certificado + 5 % del valor de materiales	Instalación + 2 días (5 / 7)	2
Mantenimiento de aire	Base según zona + Q110 × horas estimadas + Q80 de gas refrigerante	2 días	1

Ejemplos de verificación: una instalación metropolitana de 2.5 h y 5 puntos tiene 3.75 h por puntos, se facturan 4 h y cuesta Q100 + Q360 = Q460. Con materiales de Q2,000, la eléctrica certificada cuesta Q460 + Q120 + Q100 = Q680.

1.4 Reglas transversales

- Toda visita adicional a la primera genera Q40, que se registra cuando la jornada **inicia**, no cuando se asigna la orden.
- Una orden que agota sus visitas queda cancelada y no puede asignarse de nuevo.
- Cotizaciones, retiros de órdenes y operaciones rechazadas no alteran los ingresos.
- Toda operación inválida se rechaza con un mensaje claro y sin modificar el estado previo. Las entradas con formato incorrecto no terminan el programa.
- Los montos se muestran con dos decimales y los datos viven solo en memoria.

2. Decisiones que dependen de la categoría

#	Decisión	Clase que la toma	Por qué
1	Nombre de la categoría (para reportes)	Cada subclase: `categoria()`	Es una característica propia de la categoría; los reportes agrupan por este nombre sin conocer las clases
2	Costo de la orden	Cada subclase: `calcularCosto()`	Cada categoría tiene su fórmula. La eléctrica reutiliza la de Instalación con `super.calcularCosto()`
3	Horas facturables y horas por puntos	`Instalacion`	Solo existen en instalaciones; la eléctrica las hereda
4	Garantía y certificado	`InstalacionElectricaCertificada`	Solo existen en esa categoría

#	Decisión	Clase que la toma	Por qué
5	Tiempo estimado de atención	Cada subclase: `diasAtencion()`	Cada categoría tiene su propia tabla de días. La eléctrica suma 2 a lo que dice Instalación
6	Número máximo de visitas	Cada subclase: `maxVisitas()`	Es la política de visitas de cada categoría. `OrdenServicio` la usa para decidir entre pendiente y cancelada
7	Compatibilidad con la cuadrilla	Cada subclase: `esCompatibleCon(Cuadrilla)`	`MantenimientoAire` exige certificación en refrigeración; las demás aceptan cualquier cuadrilla (valor por defecto en `OrdenServicio`). La jornada solo pregunta, no conoce el motivo
8	Límite de 1.5 horas en diagnósticos	Constructor de `Diagnostico`	Es una restricción de datos exclusiva de esa categoría; así no puede existir un diagnóstico inválido
9	Puntos enteros positivos	Constructor de `Instalacion`	Dato propio de la instalación
10	Rango de BTU y zona metropolitana obligatoria	Constructor de `MantenimientoAire`	Restricciones propias del mantenimiento de aire
11	Valor de materiales positivo	Constructor de `InstalacionElectricaCertificada`	Dato propio de la eléctrica
12	Cargo por visita adicional	`OrdenServicio.cargoVisitaAdicional()` (Q40)	Hoy es igual para todas las categorías, por eso vive en la base; al ser sobrescribible, una categoría nueva podría cambiarlo sin tocar nada más

Decisiones que **no** dependen de la categoría y por eso **no** están en las subclases:

- Capacidad de la cuadrilla, unicidad de códigos y orden de las transiciones de estado: las toman `Jornada`, `TecniHogar` y `OrdenServicio` una sola vez, para todas las categorías.
- Tarifa base por zona (Q100 / Q175): la comparten Instalación y Mantenimiento de aire, así que vive una sola vez en `OrdenServicio.tarifaBasePorZona()` para no duplicar la regla.

3. Estados y transiciones

3.1 Estados de una orden

Estado actual	Operación que provoca la transición	Estado siguiente

Estado actual	Operación que provoca la transición	Estado siguiente
(no existe)	Registrar orden (aceptada la cotización)	Pendiente
Pendiente	Asignar a una jornada en preparación	Asignada
Asignada	Retirar de la jornada	Pendiente
Asignada	Iniciar la jornada	En atención (se cuenta la visita y, desde la segunda, se cobra Q40)
En atención	Registrar resultado "completada"	Completada
En atención	Registrar "visita fallida" con visitas disponibles	Pendiente (puede asignarse a otra jornada)
En atención	Registrar "visita fallida" con visitas agotadas	Cancelada

Completada y Cancelada son estados finales: desde ellos no se permite ninguna operación sobre la orden.

3.2 Estados de una jornada

Estado actual	Operación que provoca la transición	Estado siguiente
(no existe)	Crear jornada (la cuadrilla pasa a ocupada)	En preparación
En preparación	Asignar o retirar órdenes	En preparación
En preparación	Iniciar jornada (con al menos una orden)	En atención
En atención	Registrar resultado de una orden que no es la última	En atención
En atención	Registrar resultado de la última orden pendiente de resultado	Cerrada (la cuadrilla queda libre)

Asignar y retirar solo se permiten en preparación; registrar resultados solo se permite en atención.

4. Clases y responsabilidades

4.1 Visión general

Clase	Responsabilidad
`OrdenServicio` (abstracta)	Datos comunes de toda orden, su estado y sus transiciones. Declara lo que cada categoría debe decidir
`Diagnostico`, `Instalacion`, `InstalacionElectricaCertificada`, `MantenimientoAire`	Reglas propias de cada categoría: validación de sus datos, costo, tiempo, visitas y compatibilidad

Clase	Responsabilidad
`Cuadrilla`	Datos de la cuadrilla y si está libre
`Jornada`	Ciclo de vida de una jornada: asignar, retirar, iniciar, registrar resultados y cerrar. Valida capacidad, compatibilidad y estado
`Ingresos`	Único lugar donde se acumula dinero
`TecniHogar`	Fachada: guarda órdenes, cuadrillas y jornadas, garantiza la unicidad de códigos, registra ingresos y arma los reportes
`DatosIniciales`	Carga los datos de arranque sin ingresos
`MenuConsola`	Entrada y salida por consola. No contiene reglas de negocio
`Main`	Contiene `main`: crea el sistema, carga los datos y abre el menú
`Zona`, `EstadoOrden`, `EstadoJornada`, `ResultadoVisita`	Enumeraciones. Evitan comparar cadenas
`OperacionInvalidaException`	Excepción única para toda regla violada
`Validaciones`, `Formato`	Utilidades estáticas: validaciones reutilizables y formato de montos con dos decimales

4.2 `OrdenServicio` (abstracta)

Atributos (todos `private`; el estado solo cambia mediante los métodos de transición):

Atributo	Tipo	Propósito
`codigo`	`String` (final)	Identifica la orden
`cliente`	`String` (final)	Nombre del cliente
`direccion`	`String` (final)	Dirección del servicio
`horasEstimadas`	`double` (final)	Horas de trabajo estimadas; se usan para cobrar y para la capacidad de la cuadrilla
`zona`	`Zona` (final)	Metropolitana o periférica
`estado`	`EstadoOrden`	Estado actual de la orden
`visitasRealizadas`	`int`	Visitas que se han iniciado

Atributo	Tipo	Propósito
`TARIFA_BASE_METROPOLITANA`, `TARIFA_BASE_PERIFERICA`	`double` (constantes `private static`)	Q100 y Q175
`CARGO_VISITA_ADICIONAL`	`double` (constante `private static`)	Q40

Constructor: `protected OrdenServicio(String codigo, String cliente, String direccion, double horasEstimadas, Zona zona)`. Valida cada dato (no vacío, positivo, no nulo), inicializa el estado en pendiente y las visitas en 0. Es `protected` porque la clase es abstracta y solo las subclases lo invocan.

Métodos:

Método	Visibilidad	Parámetros → resultado	Propósito
`categoria()`	`public abstract`	→ `String`	Nombre de la categoría
`calcularCosto()`	`public abstract`	→ `double`	Costo total de la orden
`diasAtencion()`	`public abstract`	→ `int`	Días hábiles estimados
`maxVisitas()`	`public abstract`	→ `int`	Visitas permitidas
`esCompatibleCon(Cuadrilla)`	`public`	cuadrilla → `boolean`	Compatibilidad con la cuadrilla; por defecto `true`
`cargoVisitaAdicional()`	`protected`	→ `double`	Cargo por visita adicional; por defecto Q40
`detalleEspecifico()`	`protected`	→ `String`	Texto extra para consultas; las subclases agregan sus atributos
`porZona(double, double)` y `porZona(int, int)`	`protected final`	valor metropolitano, valor periférico → valor	Escoge un valor según la zona; evita repetir el `if` en cada subclase
`tarifaBasePorZona()`	`protected final`	→ `double`	Tarifa base compartida por Instalación y Mantenimiento
`validarAsignable()`	package	→ nada	Lanza excepción si la orden no está pendiente (condición 3 de asignación)
`marcarAsignada()`	package	→ nada	Pendiente → Asignada
`marcarPendiente()`	package	→ nada	Asignada → Pendiente (retiro)

Método	Visibilidad	Parámetros → resultado	Propósito
`iniciarVisita()`	package	→ `double`	Asignada → En atención; suma una visita y devuelve el cargo (0 la primera vez, Q40 después)
`registrarResultado(ResultadoVisita)`	package	resultado → nada	En atención → Completada, Pendiente o Cancelada según el resultado y `maxVisitas()`
Consultores `getCodigo()`, `getCliente()`, `getDireccion()`, `getHorasEstimadas()`, `getZona()`, `getEstado()`, `getVisitasRealizadas()`	`public`	→ valor	Solo lectura; no hay setters
`toString()`	`public`	→ `String`	Descripción de la orden

Las transiciones son de paquete porque las dispara únicamente `Jornada`, que coordina el ciclo de vida; así ninguna otra clase puede alterar el estado de una orden.

4.3 Subclases de categoría

`Diagnostico` (extiende `OrdenServicio`)

- Atributos: constantes `HORAS_MAXIMAS` (1.5, `public static final`), `COSTO_METROPOLITANA` (150), `COSTO_PERIFERICA` (225) y `MAX_VISITAS` (3), todas `private static final` salvo la primera. No agrega atributos de instancia.
- Constructor: `Diagnostico(codigo, cliente, direccion, horasEstimadas, zona)`. Llama a `super(...)` y luego rechaza horas > 1.5.
- Sobrescribe: `categoria()`, `calcularCosto()` (`porZona(150, 225)`), `diasAtencion()` (`porZona(1, 2)`), `maxVisitas()` (3).

`Instalacion` (extiende `OrdenServicio`)

Miembro	Tipo	Visibilidad	Propósito
`puntos`	`int` (final)	`private`	Puntos a instalar (entero positivo)
`TARIFA_POR_HORA`, `HORAS_POR_PUNTO`, `MAX_VISITAS`	constantes	`private static final`	Q90, 0.75 y 3
Constructor `Instalacion(codigo,`		`public`	Llama a `super(...)` con los datos comunes y valida los puntos

Miembro	Tipo	Visibilidad	Propósito
`cliente, direccion, horasEstimadas, zona, puntos)`			
`getPuntos()`	→ `int`	`public`	Consulta
`horasPorPuntos()`	→ `double`	`public`	puntos × 0.75
`horasFacturables()`	→ `int`	`public`	Techo del mayor entre horas estimadas y horas por puntos (con tolerancia de 1e-9 para errores de punto flotante)
`categoria()`, `calcularCosto()`, `diasAtencion()`, `maxVisitas()`		`public` (sobrescritos)	Tarifa base + Q90 × horas facturables; 3 / 5 días; 3 visitas
`detalleEspecifico()`		`protected` (sobrescrito)	Agrega los puntos

`InstalacionElectricaCertificada` (extiende `Instalacion`)

Miembro	Tipo	Visibilidad	Propósito
`valorMateriales`	`double` (final)	`private`	Valor declarado de los materiales
`CARGO_CERTIFICADO`, `PORCENTAJE_GARANTIA`, `DIAS_ADICIONALES`, `MAX_VISITAS`	constantes	`private static final`	Q120, 0.05, 2 y 2
Constructor `(codigo, cliente, direccion, horasEstimadas, zona, puntos, valorMateriales)`		`public`	Llama a `super(...)` (Instalación) y valida los materiales
`getValorMateriales()`	→ `double`	`public`	Consulta
`calcularGarantia()`	→ `double`	`public`	5 % de los materiales
`calcularCosto()`	→ `double`	`public` (sobrescrito)	`super.calcularCosto()` + Q120 + garantía
`diasAtencion()`	→ `int`	`public` (sobrescrito)	`super.diasAtencion()` + 2
`maxVisitas()`, `categoria()`, `detalleEspecifico()`		sobrescritos	2 visitas; nombre propio; agrega los materiales

`MantenimientoAire` (extiende `OrdenServicio`)

Miembro	Tipo	Visibilidad	Propósito
`btu`	`int` (final)	`private`	Capacidad del equipo
`BTU_MINIMO`, `BTU_MAXIMO`	`int`	`public static final`	9,000 y 60,000
`TARIFA_POR_HORA`, `CARGO_GAS`, `MAX_VISITAS`	constantes	`private static final`	Q110, Q80 y 1
Constructor `(codigo, cliente, direccion, horasEstimadas, zona, btu)`		`public`	Llama a `super(...)`; rechaza zona periférica y BTU fuera de rango
`getBtu()`	→ `int`	`public`	Consulta
`calcularCosto()`	→ `double`	`public` (sobrescrito)	Tarifa base + Q110 × horas estimadas + Q80
`esCompatibleCon(Cuadrilla)`	→ `boolean`	`public` (sobrescrito)	Solo cuadrillas certificadas en refrigeración
`categoria()`, `diasAtencion()`, `maxVisitas()`, `detalleEspecifico()`		sobrescritos	Nombre propio; 2 días siempre; 1 visita; agrega BTU

4.4 `Cuadrilla`

Miembro	Tipo	Visibilidad	Propósito
`codigo`	`String` (final)	`private`	Identificador único
`capacidadHoras`	`double` (final)	`private`	Horas máximas por jornada
`certificadaRefrigeracion`	`boolean` (final)	`private`	Si puede atender mantenimiento de aire
`libre`	`boolean`	`private`	`false` mientras tenga una jornada sin cerrar
Constructor `Cuadrilla(String codigo, double capacidadHoras, boolean certificadaRefrigeracion)`		`public`	Valida código y capacidad; inicia libre
`ocupar()`	→ nada	package	La llama `Jornada` al crearse; falla si ya está ocupada
`liberar()`	→ nada	package	La llama `Jornada` al cerrarse

Miembro	Tipo	Visibilidad	Propósito
`estaLibre()`, `getCodigo()`, `getCapacidadHoras()`, `isCertificadaRefrigeracion()`	→ valor	`public`	Consultas

4.5 `Jornada`

Miembro	Tipo	Visibilidad	Propósito
`codigo`	`String` (final)	`private`	Identificador generado (J-001, J-002...)
`cuadrilla`	`Cuadrilla` (final)	`private`	Cuadrilla que atiende la jornada
`ordenes`	`List<OrdenServicio>`	`private`	Órdenes asignadas
`resultados`	`Map<OrdenServicio, ResultadoVisita>`	`private`	Resultado registrado por orden. Se lleva aquí y no se deduce del estado de la orden, porque una orden fallida vuelve a pendiente y puede entrar a otra jornada mientras esta sigue abierta
`estado`	`EstadoJornada`	`private`	Estado actual
Constructor `Jornada(String codigo, Cuadrilla cuadrilla)`		`public`	Valida los datos, **ocupa la cuadrilla** (falla si no está libre) y queda en preparación
`asignar(OrdenServicio)`	→ nada	`public`	Valida estado de la jornada, estado de la orden, compatibilidad y capacidad, **en ese orden y antes de modificar nada**; luego asigna
`retirar(OrdenServicio)`	→ nada	`public`	Solo en preparación; la orden vuelve a pendiente
`iniciar(Ingresos)`	→ nada	`public`	Solo en preparación y no vacía; inicia la visita de cada orden y **registra en `Ingresos` el cargo por visita adicional**
`registrarResultado(OrdenServicio, ResultadoVisita)`	→ nada	`public`	Solo en atención; registra el resultado y, si fue el último, cierra la jornada y libera la cuadrilla

Miembro	Tipo	Visibilidad	Propósito
`exigirEstado(EstadoJornada, String)`	→ nada	`private`	Validación de estado reutilizada por las operaciones anteriores
`horasAsignadas()`	→ `double`	`public`	Suma de horas **estimadas** (no facturables)
`capacidadDisponible()`	→ `double`	`public`	Capacidad de la cuadrilla − horas asignadas
`getCodigo()`, `getCuadrilla()`, `getEstado()`, `getOrdenes()`	→ valor	`public`	Consultas; `getOrdenes()` devuelve una lista no modificable

4.6 `Ingresos`

Miembro	Tipo	Visibilidad	Propósito
`porRegistro`	`Map<String, Double>`	`private`	Ingreso por registro, por categoría
`porVisitasAdicionales`	`Map<String, Double>`	`private`	Ingreso por visitas adicionales, por categoría
`registrarPorRegistro(String categoria, double monto)`	→ nada	package	La llama solo `TecniHogar.registrarOrden()`
`registrarVisitaAdicional(String categoria, double monto)`	→ nada	package	La llama solo `Jornada.iniciar()`
`totalPorRegistro()`, `totalPorVisitasAdicionales()`, `total()`	→ `double`	`public`	Totales
`registroDe(String)`, `visitasAdicionalesDe(String)`, `totalDe(String)`	→ `double`	`public`	Ingreso de una categoría
`categorias()`	→ `Set<String>`	`public`	Categorías con ingresos

Los métodos que modifican son de paquete y solo se invocan desde los dos puntos descritos. Por eso las cotizaciones (que nunca registran), los retiros y las operaciones rechazadas (que fallan antes de llegar a este punto) no pueden alterar los ingresos.

4.7 `TecniHogar`

Miembro	Tipo	Visibilidad	Propósito
`ordenes`, `cuadrillas`, `jornadas`	`Map<String, ...>`	`private`	Colecciones indexadas por código en mayúsculas (la unicidad no distingue mayúsculas)
`ingresos`	`Ingresos` (final)	`private`	Libro de ingresos
Constructor		`public` (por defecto)	Crea las colecciones y el libro vacíos
`registrarOrden(OrdenServicio)`	→ nada	`public`	Guarda la orden y registra su costo como ingreso por registro; si el código está repetido falla antes de cobrar
`cargarOrdenInicial(OrdenServicio)`	→ nada	package	Guarda sin cobrar; solo para los datos iniciales
`registrarCuadrilla(Cuadrilla)`	→ nada	`public`	Guarda la cuadrilla con código único
`crearJornada(String codigoCuadrilla)`	→ `Jornada`	`public`	Crea la jornada con código generado
`asignarOrden`, `retirarOrden`, `iniciarJornada`, `registrarResultado`	→ nada	`public`	Buscan jornada y orden por código y delegan en `Jornada`
`buscarOrden`, `buscarCuadrilla`, `buscarJornada`	→ objeto	`public`	Búsqueda; lanzan excepción si no existe
`conteoPorCategoriaYEstado()`	→ `Map<String, Map<EstadoOrden, Integer>>`	`public`	Reporte de órdenes; agrupa con `categoria()`
`categoriasConocidas()`	→ `Set<String>`	`public`	Categorías presentes; no hay lista fija
`jornadasNoCerradas()`	→ `List<Jornada>`	`public`	Reporte de jornadas abiertas
`getIngresos()`, `getOrdenes()`, `getCuadrillas()`, `getJornadas()`	→ valor	`public`	Consultas (colecciones no modificables)

4.8 `MenuConsola`, `Main` y `DatosIniciales`

- `MenuConsola` tiene `empresa` (`TecniHogar`, privado) y `entrada` (`Scanner`, privado). Su constructor recibe la empresa. `ejecutar()` repite el menú hasta que el usuario sale y atrapa `OperacionInvalidaException` para mostrar el mensaje sin terminar. Los métodos de lectura (`leerEntero`, `leerDecimal`, `leerTexto`, `leerSiNo`, `leerZona`) convierten un formato incorrecto en una excepción de negocio.
- `Main` contiene el método `main`: crea `TecniHogar`, llama a `DatosIniciales.cargar(empresa)` y ejecuta `new MenuConsola(empresa).ejecutar()`.
- `DatosIniciales.cargar(TecniHogar)` es estático y carga nueve órdenes pendientes (al menos dos por categoría, incluida una instalación cuyas horas por puntos superan las estimadas) y tres cuadrillas (una certificada y dos no). Usa `cargarOrdenInicial`, por lo que no registra ingresos.

4.9 Cómo reciben sus valores iniciales los objetos

- **Todos los datos entran por el constructor** y los atributos son `final` cuando no cambian. No hay setters, así que ningún objeto queda a medio construir.
- **Colaboración dentro de la jerarquía:** el constructor de cada subclase llama primero a `super(...)` con los datos que corresponden a la clase padre y después valida y guarda los suyos. En la cadena `InstalacionElectricaCertificada` → `Instalacion` → `OrdenServicio`, la eléctrica pasa `codigo, cliente, direccion, horasEstimadas, zona` y `puntos` a `Instalacion`; esta pasa los cinco comunes a `OrdenServicio`, que valida y guarda los datos comunes y deja el estado en pendiente con 0 visitas. Cada clase valida solo lo suyo.
- **Un dato inválido impide que el objeto exista.** Como la excepción sale del constructor y el objeto aún no se guardó en ninguna colección, no hay estado previo que corregir. Esto también hace que cotizar sea seguro: una cotización es un objeto que nunca se registra.
- Los demás objetos se inicializan igual: `Cuadrilla` empieza libre; `Jornada` empieza en preparación, sin órdenes ni resultados, y ocupa la cuadrilla al crearse; `Ingresos` empieza vacío.

5. Justificación de la jerarquía: instalación eléctrica certificada

Decisión: `InstalacionElectricaCertificada` **extiende** `Instalacion`, que a su vez extiende `OrdenServicio`.

Por qué: el enunciado la define como "una instalación que además requiere" certificado y garantía, y dice que se calcula igual que una instalación y se le suman cargos. Eso es una relación "es un": toda instalación eléctrica certificada es una instalación y puede usarse en cualquier lugar donde se espere una. Hereda los puntos, las horas por puntos, las horas facturables y el cálculo base, y solo agrega lo propio (materiales, certificado, garantía) y sobrescribe lo que cambia (costo, días y visitas). Una orden de este tipo se puede asignar a cualquier jornada y aparece en los reportes igual que las demás, sin código especial.

Qué se ganaría o perdería con alternativas:

Alternativa	Qué se pierde	Qué se gana
Hermana de `Instalacion`	Habría que repetir los puntos, las horas por puntos,	Independencia

Alternativa	Qué se pierde	Qué se gana
(extiende directamente `OrdenServicio`)	las horas facturables y la tarifa base. Si cambia la regla de instalación (por ejemplo, de 0.75 a 0.80 horas por punto) habría que corregirla en dos clases y es fácil que queden distintas. Se viola la regla de no duplicar reglas entre categorías	total: un cambio en `Instalacion` no podría afectarla
Una sola clase `Instalacion` con un atributo booleano "certificada"	Aparecerían `if (certificada)` en el costo, los días y las visitas: justo lo que el enunciado prohíbe. Además `valorMateriales` existiría en instalaciones normales con un valor sin sentido. Una categoría nueva obligaría a editar la clase	Menos clases
Composición (la eléctrica contiene una instalación y delega)	Habría que reenviar a mano todos los métodos y no podría usarse como `Instalacion` sin código adicional	Evitar el acoplamiento con `Instalacion`

La herencia tiene un costo conocido y aceptable: cualquier cambio de `Instalacion` se propaga a la eléctrica. Aquí es lo que se desea, porque el enunciado dice que se calcula igual que una instalación.

6. Impacto de agregar una nueva categoría

Ejemplo: instalación de paneles solares, con sus reglas de costo, tiempo, cuadrilla y visitas.

Clases que hay que crear:

- Una clase nueva que extienda `OrdenServicio` (o `Instalacion` si fuera una instalación en sentido estricto) y que valide en su constructor los datos propios, implemente `categoria()`, `calcularCosto()`, `diasAtencion()` y `maxVisitas()`, y, si tiene restricción de cuadrilla, sobrescriba `esCompatibleCon(Cuadrilla)`. Si su cargo por visita adicional fuera distinto, sobrescribiría `cargoVisitaAdicional()`.

Clases que hay que modificar:

- `MenuConsola`: agregar la opción de categoría y la lectura de sus datos propios en `leerOrden()`. Es el único punto de la interfaz que conoce las categorías, porque ahí se decide qué constructor llamar.
- `DatosIniciales` (opcional): para cargar órdenes de ejemplo de la categoría nueva.

Clases que no cambian:

- `OrdenServicio` y las otras categorías: la nueva clase solo se agrega.
- `Jornada`: valida la compatibilidad preguntando `esCompatibleCon(...)` y la capacidad con las horas estimadas, sin conocer categorías. La política de visitas la resuelve cada orden en `registrarResultado()`.
- `TecniHogar`, `Ingresos` y `Cuadrilla`: no mencionan ninguna categoría concreta.

Reportes y jornadas: ninguno tiene que cambiar. Los reportes agrupan por el texto que devuelve `categoria()` y obtienen las categorías existentes de las propias órdenes, de modo que la nueva aparece

sola en el conteo por categoría y estado y en los ingresos. Las jornadas, el cargo por visita adicional y los ingresos funcionan igual para cualquier orden.

Un caso que sí obligaría a tocar más clases: si la restricción de cuadrilla de la categoría nueva dependiera de un dato que hoy no existe (por ejemplo, una cuadrilla certificada en energía solar), habría que agregar ese atributo a `Cuadrilla` y a su registro en el menú. La decisión de compatibilidad seguiría en la nueva categoría y `Jornada` no cambiaría, pero `Cuadrilla` sí.

7. Diseño

